

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1. In a particle system designed with Processing using Python, your objective is to implement the drawTrail function for the Particle class. This function should draw a trail behind a particle to visually indicate its path across the canvas. Assume each particle has a list of its previous positions stored in self.trail, where each position is a PVector. The length of the trail should not exceed a specified maximum length to prevent the list from growing indefinitely. How should the drawTrail function be implemented?

1 / 3 point

```
1 class Particle:
2     def __init__(self, x, y):
3         self.position = PVector(x, y)
4         self.trail = []
5         self.trailMaxLength = 10 # Maximum trail length
6
7     def update(self):
8         self.position.add(PVector(random(), random()))
9         self.trail.append(self.position.copy())
10        # Ensure the trail does not exceed the maximum length
11        if len(self.trail) > self.trailMaxLength:
12            self.trail.pop(0)
13
14    def drawTrail(self):
15        # Function implementation needed here
16
17    def setup(self):
18        size(400, 400)
19        global p
20        p = Particle(200, 200)
21
22    def draw():
23        background(255)
24        p.update()
25        p.drawTrail()
26
```

☒

```
1 def drawTrail(self):
2     for i in range(len(self.trail) - 1):
3         strokeWeight(2)
4         stroke(255, 0, 0, 255 - 1 * (255 / self.trailMaxLength))
5         line(self.trail[i].x, self.trail[i].y, self.trail[i+1].x, self.trail[i+1].y)
6
```

☒ **Correct**
Correct and creative. This solution not only draws the trail but also varies the stroke's transparency based on the position's age within the trail, creating a visual effect where older segments are fainter. This adds a nice touch by visually indicating the direction of movement through the fading trail.

☐

```
1 def drawTrail(self):
2     beginShape()
3     for pos in self.trail:
4         vertex(pos.x, pos.y)
5     endShape()
6
```

☐

```
1 def drawTrail(self):
2     for pos in self.trail:
3         point(pos.x, pos.y)
4
```

☐

```
1 def drawTrail(self):
2     stroke(0)
3     noFill()
4     for i in range(1, len(self.trail)):
5         ellipse(self.trail[i].x, self.trail[i].y, 5, 5)
6
```

☒

```
1 def drawTrail(self):
2     for i in range(1, len(self.trail)):
3         stroke(0)
4         line(self.trail[i-1].x, self.trail[i-1].y, self.trail[i].x, self.trail[i].y)
5
```

☒ **Correct**
Correct. This implementation correctly iterates over the list of previous positions, drawing a line between consecutive positions. It effectively visualizes the trail as a series of lines that represent the particle's path, adhering to the requirement.

2. In a particle system developed with Processing using Python, your goal is to simulate gravity affecting the particles. Each particle is represented by a class that includes position, velocity, and acceleration attributes. You need to implement the applyGravity function within the Particle class that applies a gravitational force to the particle's acceleration, simulating the effect of gravity pulling the particle downward. How should the applyGravity function be written?

1 / 3 point

```
1 class Particle:
2     def __init__(self, x, y):
3         self.position = PVector(x, y)
4         self.velocity = PVector(0, 0)
5         self.acceleration = PVector(0, 0)
6
7     def update(self):
8         self.velocity.add(self.acceleration)
9         self.position.add(self.velocity)
10        # Reset acceleration each frame
11        self.acceleration.mult(0)
12
13    def applyGravity(self):
14        # Function implementation needed here
15
16    def display(self):
17        ellipse(self.position.x, self.position.y, 10, 10)
18
19    def setup(self):
20        size(400, 400)
21        global p
22        p = Particle(200, 200)
23
24    def draw():
25        background(255)
26        p.applyGravity()
27        p.update()
28        p.display()
29
```

☒

```
1 def applyGravity(self):
2     gravity = PVector(0, 9.8)
3     self.acceleration.add(gravity)
4
```

☐

```
1 def applyGravity(self):
2     self.position.y += 0.1
3
```

☐

```
1 def applyGravity(self):
2     self.velocity.y += 0.1
3
```

☐

```
1 def applyGravity(self):
2     gravity = PVector(0, 9.8)
3     self.acceleration.add(gravity)
4
```

☐

```
1 def applyGravity(self):
2     gravity = PVector(0, 1)
3     self.acceleration = gravity
4
```

☒ **Correct**
Correct. This solution accurately represents the effect of gravity by creating a downward force (positive y-direction). Adding this force to the particle's acceleration simulates gravity's influence effectively, adhering to the principles of Newtonian physics.

3. In a particle system using Processing with Python, each particle has attributes for position, velocity, and acceleration. Your task is to implement the applyForce function within the Particle class. This function should take a PVector representing an arbitrary force (e.g., wind, attraction) and apply it to the particle's acceleration, allowing for the simulation of various environmental effects. How should the applyForce function be implemented?

1 / 3 point

```
1 class Particle:
2     def __init__(self, x, y):
3         self.position = PVector(x, y)
4         self.velocity = PVector(0, 0)
5         self.acceleration = PVector(0, 0)
6
7     def update(self):
8         self.velocity.add(self.acceleration)
9         self.position.add(self.velocity)
10        # Reset acceleration each frame
11        self.acceleration.mult(0)
12
13    def applyForce(self, force):
14        # Function implementation needed here
15
16    def display(self):
17        ellipse(self.position.x, self.position.y, 10, 10)
18
19    def setup(self):
20        size(400, 400)
21        global p
22        p = Particle(200, 200)
23
24    def draw():
25        background(255)
26        wind = PVector(0.1, 0)
27        p.applyForce(wind)
28        p.update()
29        p.display()
30
```

☒

```
1 def applyForce(self, force):
2     self.acceleration.add(force)
3
```

☒ **Correct**
Correct. This solution is accurate and follows Newton's second law of motion, assuming a unit mass for simplicity. By adding the force directly to the acceleration, it correctly simulates the influence of an external force on the particle, allowing for realistic behavior in the particle system.

☐

```
1 def applyForce(self, force):
2     self.velocity.add(force)
3
```

☒

```
1 def applyForce(self, force):
2     accelerationChange = PVector.div(force, self.mass)
3     self.acceleration.add(accelerationChange)
4
```

☒ **Correct**
Potentially Correct. This approach is more physically accurate, as it divides the force by the particle's mass to determine the change in acceleration, adhering to $F=ma$. However, the skeleton code doesn't define self.mass, so this solution would require adding a mass attribute to the Particle class for it to be correct.

☐

```
1 def applyForce(self, force):
2     self.acceleration = force
3
```

☐

```
1 def applyForce(self, force):
2     self.position.add(force)
3
```